

A/B Testing Report: Quadruped Objective Function Comparison

Linus Mehring | Spring Evolutionary Robotics | April 29, 2026

Section 1: Goals

The main project for this deliverable was evolving a quadruped robot controller using a parallel hill climber. The robot is a four-legged body simulated in PyBullet, and the goal was to evolve its neural network weights so it performs some interesting behavior. The broader question I wanted to explore was: does the choice of fitness function actually matter for the quality and character of the evolved behavior?

To set up the A/B test, I defined two variants based on what the fitness function was rewarding. Variant A uses a simple measure of airborne time -- specifically, the longest contiguous segment of timesteps during which the torso was above a threshold height. The idea is that a robot with a high score in this variant learned to jump or at least get off the ground for a sustained period. Variant B uses a compound objective: the product of horizontal distance traveled and that same airborne duration. The reasoning there is that just floating in place isn't all that impressive, so multiplying by horizontal distance should push evolution toward robots that actually move forward while also getting air.

The question I was trying to answer was: does the compound objective (Variant B) produce a meaningfully better or more capable robot than the simpler airborne-only objective (Variant A)? And relatedly, does either variant actually show signs of evolution at all -- i.e., do robots get better over time, or does it look like random noise?

Section 2: Implementation Details

The codebase uses a parallel hill climber architecture. Each generation, every parent robot spawns a mutated child, both are evaluated, and the better one survives. The population size was 10 and each run went for 10 generations, so each full run involved 110 total evaluations (10 initial parents plus 10 per generation).

To support A/B testing, I added a `fitnessMode` parameter that threads through the entire call stack -- from `search.py` at the top level, down through `PARALLEL_HILL_CLIMBER`, `SOLUTION`, `SIMULATION`, and finally to `robot.py` where the actual fitness computation happens. Each simulation now records torso height at every timestep, and at the end computes three metrics: the longest contiguous airborne segment, total horizontal displacement, and peak torso height. These get written to per-simulation stats files that the parent process reads back in.

For Variant A, the fitness is set directly to `longestFlight`. For Variant B, fitness is `horizontalDistance x longestFlight`. The mutation strategy was left the same across both variants -- a single random

synapse weight gets perturbed -- so any difference in outcomes can be attributed to the fitness function rather than the search process.

To run the actual A/B test, I executed three independent runs of each variant, keeping population size and generation count identical, and recorded the best individual's stats from each run. This gives enough data to get a rough sense of whether one variant consistently outperforms the other.

Section 3: Results

Across three independent runs, both variants showed clear signs of evolution -- robots improved over generations rather than staying flat or oscillating randomly. The parallel hill climber's selection pressure consistently produced individuals with higher fitness than what was seen in generation zero.

Run	A Fitness	A Flight	A Dist.	A Max H	B Fitness	B Flight	B Dist.	B Max H
1	124.000	124	1.776	1.370	251.823	64	3.935	1.173
2	114.000	114	0.681	1.435	303.907	92	3.303	1.309
3	123.000	123	5.668	1.301	337.008	97	3.474	1.492

Metric	Mode A	Mode B
Mean Fitness	120.333	297.579
Mean Longest Flight (timesteps)	120.333	84.333
Mean Horizontal Distance	2.708	3.571
Mean Max Height	1.369	1.325

The fitness values aren't directly comparable between modes since they measure different things, but looking at the underlying metrics tells a clear story. Variant B consistently produced robots that traveled farther -- mean horizontal distance of 3.571 vs 2.708 for Variant A. Variant A robots had longer airborne durations on average (120 timesteps vs 84), which makes sense since that's all it's optimizing for. The interesting result is that Variant B's compound pressure got robots off the ground for 84 timesteps and moved them forward 3.5 units, whereas Variant A robots sometimes flew for longer but barely went anywhere (Run 2: 114 timesteps, only 0.681 horizontal distance).

In terms of which variant is better, it depends on what you care about. If the goal is a robot that jumps -- Variant A wins. If the goal is a robot that locomotes in an interesting way -- Variant B is clearly the better objective.

Section 4: Reflection

The surprisingly difficult part was getting the fitness computation right. Specifically, defining airborne in a way that was meaningful took some thought. The naive approach of just checking whether height is above some fixed threshold has a problem: you have to pick the threshold, and if it's too low every robot looks like it's flying, and if it's too high nothing qualifies. I ended up using a threshold relative to the robot's resting height, which worked reasonably well but is still a bit hacky.

The surprisingly easy part was threading the fitness mode through the codebase. Because the architecture was already modular -- with clean separation between simulation, solution, and the hill climber -- adding a single fitnessMode parameter that flows down the call stack took maybe 20 minutes total. The existing structure made it almost trivially easy to add a new objective without restructuring anything.

If I had another year on this project, the most interesting direction would be evolving the robot's body morphology alongside the controller weights. Right now the body is fixed -- same four-legged shape every run -- and only the neural network weights evolve. Allowing limb lengths, joint placements, or even the number of legs to vary would make the search space much richer and probably produce more interesting behaviors. A natural extension to the A/B test would be asking: does morphological evolution benefit more from Variant A or B?

In terms of Pyrosim features, the thing I most wanted during this project was better built-in support for contact sensing. Right now detecting whether a foot is on the ground requires querying PyBullet's contact points at every step, which is doable but not exposed cleanly through Pyrosim. If Pyrosim had a `Get_Contact_State(bodyID, linkID)` call, writing more nuanced fitness functions -- like rewarding a specific gait pattern -- would be a lot more accessible.

Extending the project to morphological evolution would probably be moderately hard. The search space is larger and the simulation time per evaluation would increase, but the hill climber architecture already handles the loop -- you would mostly be adding a genome representation for body shape. The genuinely hard part would be handling the edge cases in procedural URDF generation, particularly making sure mutated morphologies stay physically valid (no intersecting links, reasonable joint ranges, etc.). That kind of constraint-aware mutation logic would take significant engineering effort to get right.